LFTC: Lab 3 - Documentation

> ***Statement**: For the domain specific programming language from Lab 1, perform lexical analysis. Choose between using the tool lex / flex or Python (mandatory using regex)*

Tools and Source Code Structure

The chosen IDE was CLion and the main implementation language C. The Symbol Table was implemented as a Binary Search Tree (BST). Both the ST and PIF were written in C.

For the scanner, Flex was used. The rules were written in the `scanner.l` file and WinFlexBison automatically generated `lex.yy.c` from it.

The source program, which are in `.txt` format are passed to the program as command-line arguments.

Token Detection

The scanner recognizes *keywords, operators, separators, identifiers and integers(constants).* Spaces, tabs and line breaks are ignoredand not taken as tokens.

the `tokens.txt` file contains the list of tokens which `tokens.c` uses in order to generate an absolute token code for each of the tokens. Codes 0 and 1 are reserved for identifiers and constants, so the token code is conmputed as `i+2,` where *i = the position in the token list.*

ST

The ST is implemented in C using a BST, storing identifiers and constants as nodes in lexicograpical (alphabetical) order, by comparing the symbols with using `strcmp().`

Each node of the BST keeps a `position` = the symbol's ID in the Symbol Table, used as a reference in the PIF, starting from 1. Position 0 is reserved for symbols that dont appear in the ST. The position is assigned at insertion and it's independent of the node's actual location in the BST

The operations are:

- `init:` creates an empty ST
- `find:` returns the symbol's position **or** `-1` if it is the symbol isn't in the ST
- `insert:` adds a new symbol **or** returns its existing position
- `print:` displays the table using inorder traversal
- `destroy:` frees the allocated memory

PIF

`PIF` is an array of `PIFEntries.` A `PIFEntry = (token code, position in ST)`

- `token code` = what type of token it is (we use codes just for convetion, could've used words as well) (the token code wsa generated by `tokens.c`)
- `position in ST` = which of the identifiers/constants it is

PIF keeps each occurrence of a token, in the program's order, in comparison with the Symbol Table (ST), which doesn't keep duplicates

Testing

The scanner was tested on both programs from Lab 2, that is `program1.txt` and `program2.txt`. Both tests displayed the desired result.

Lexical Errors testing

In addition, a test with lexical errors was conceived in order to test the lexical conventions written as rules in `scanner.l:` if the scanner correctly rejects numbers with leading zeros, digits followed by letters/symbols and any other single unexpected characters.

We tested with `012`, `12abc` and `@`. All three errors were reported on the correct lines, and the invalid elements were excluded from ST and PIF. The scanner checks lexical rules only, not syntax or program logic.

Example output

for `input.txt:`

```
int quantity;
quantity = 12;
print(quantity);
```

`scanner.l`'s output:

```
KEYWORD: int
IDENTIFIER: quantity, ST position: 1
SEPARATOR: ;
IDENTIFIER: quantity, ST position: 1
OPERATOR: =
CONSTANT: 12, ST position: 2
SEPARATOR: ;
KEYWORD: print
SEPARATOR: (
IDENTIFIER: quantity, ST position: 1
SEPARATOR: )
SEPARATOR: ;
```

`Symbol Table:`

```
Symbol Table:
Position   Symbol
2           12
1           quantity
```

`PIF:`

```
PIF:
Token code   ST position
2             0
0             1
25             0
0             1
18             0
1             2
25             0
4             0
27             0
0             1
28             0
25             0
```